

CDK Global -- Databricks Learning Plan

Prepared for: CDK Global Engineering Team **Date:** February 2026 **Instructor:** Zach Jacobson, Databricks

Overview

This learning plan contains two hands-on lesson plans designed to accelerate CDK Global's adoption of Databricks AI capabilities. Each session blends lecture, live coding, and interactive exercises using CDK-relevant scenarios (dealer management, lender approval workflows, product feedback categorization).

Lesson Plan 1: Day in the Life of Deploying Agents on Databricks

Duration: 3 hours (with breaks) **Format:** Lecture + Live Demo + Hands-on Lab **Audience:** Data Engineers, ML Engineers, Platform Engineers

Learning Objectives

By the end of this session, participants will be able to:

- 1 Understand the full lifecycle of an AI agent on Databricks -- from development to production
 - 2 Build a simple agent using the `ResponsesAgent` pattern with MLflow 3
 - 3 Integrate Unity Catalog Functions and Vector Search as agent tools
 - 4 Log, register, and deploy an agent to a Model Serving endpoint
 - 5 Evaluate agent quality using MLflow GenAI evaluation (`mlflow.genai.evaluate()`)
 - 6 Monitor a deployed agent using Lakehouse Monitoring and inference tables
-

Module 1: The Agent Landscape on Databricks (30 min -- Lecture)

Key Concepts:

- **What is an AI agent?** -- An LLM-powered system that can reason, use tools, and take actions
- **Agent Framework on Databricks** -- MLflow 3 `ResponsesAgent`, tool integration, model serving
- **Agent Bricks vs. Custom Agents** -- When to use pre-built bricks (Knowledge Assistants, Genie Spaces, Multi-Agent Supervisors) vs. building from scratch
- **Unity Catalog as the governance backbone** -- All models, functions, and data governed in one place

CDK Relevance:

- Agents that help dealers answer questions about lender approval criteria
- Agents that categorize product feedback and route to appropriate teams
- Multi-agent systems that combine document Q&A with SQL-based analytics

Discussion Points:

- Where does CDK see agent opportunities in their dealer management platform?
 - What existing processes could benefit from AI augmentation?
-

Module 2: Building an Agent from Scratch (45 min -- Live Coding)

Architecture Overview:

```
User Query --> ResponsesAgent --> LLM (Foundation Model)
|
+--> Tool: UC Function (lookup_dealer_info)
+--> Tool: Vector Search (dealer_policy_docs)
+--> Tool: UC Function
(submit_lender_application)
```

Step-by-Step Walkthrough:

1 Create the agent file (agent.py)

- Import ResponsesAgent from mlflow.pyfunc
- Define the agent class with `__init__` and `predict` methods
- Use `self.create_text_output_item(text, id)` for responses (never raw dicts)

2 Define tools with Unity Catalog Functions

- Create a UC function for dealer info lookup via `CREATE FUNCTION`
- Create a UC function that wraps business logic (e.g., lender eligibility check)
- Reference tools in the agent via `resources` parameter

3 Add Vector Search for RAG

- Create a Vector Search index from dealer policy documents
- Use managed embeddings with `databricks-gte-large-en`
- Query the index within the agent's tool-calling loop

4 Test locally with `mlflow.models.predict()`

Key Code Pattern -- ResponsesAgent:

```
from mlflow.pyfunc import ResponsesAgent, ResponsesAgentResponse
class DealerAssistantAgent(ResponsesAgent):
    def __init__(self):
        # Initialize LLM client, tools, etc.
        ...
    def predict(self, messages, context=None, custom_inputs=None):
        # Agent reasoning loop
        response = self.llm.chat(messages=messages,
tools=self.tools)
        # CORRECT output format -- always use helper methods
        return ResponsesAgentResponse(

output=[self.create_text_output_item(text=response.content,
id="msg_1")]
)
```

Common Pitfalls (from GOTCHAS):

- Never return raw dicts in output -- always use `self.create_text_output_item()`
 - Specify exact package versions in `pip_requirements` when logging
 - Use `resources` parameter in `log_model` for automatic credential passthrough
-

Module 3: Logging, Registering, and Deploying (30 min -- Live Coding)

Step 1: Log the model to MLflow

```
import mlflow
with mlflow.start_run():
    model_info = mlflow.pyfunc.log_model(
        artifact_path="agent",
        python_model="agent.py",
        registered_model_name="main.agents.dealer_assistant",
        pip_requirements=["mlflow==3.6.0", "databricks-langchain",
"langgraph==0.3.4"],
        resources=[
            {"serving_endpoint": {"name":
"databricks-meta-llama-3-3-70b-instruct"}},
            {"vector_search_index": {"name":
"main.docs.dealer_policy_index"}},
            {"sql_warehouse": {"name": "Shared SQL Warehouse"}}
        ]
    )
```

Step 2: Deploy via async job (recommended for production)

- Create a Databricks Job that calls `deploy_model_serving_endpoint`
- Use async deployment to avoid timeouts (~15 min for new endpoints)
- Poll endpoint status with `get_serving_endpoint_status`

Step 3: Query the deployed endpoint

```
query_serving_endpoint(
    name="dealer-assistant-endpoint",
    messages=[{"role": "user", "content": "What lenders approve
applications with credit scores below 650?"}]
)
```

Deployment Checklist:

- [] Agent tested locally with `mlflow.models.predict()`
 - [] Model logged with correct `pip_requirements` and `resources`
 - [] Registered in Unity Catalog (`catalog.schema.model_name`)
 - [] Endpoint created and status is `READY`
 - [] End-to-end query returns expected response
-

Module 4: Evaluating Agent Quality (30 min -- Lecture + Demo)

MLflow GenAI Evaluation Workflow:

Trace Analysis --> Dataset Building --> Scorer Creation --> Evaluation

Key Concepts:

- 1 **Build evaluation datasets** from production traces or manual curation
 - Format: `{"inputs": {"query": "..."}, "expectations": {"expected_response": "..."}}`

- Use `mlflow.genai.evaluate()` (NOT `mlflow.evaluate()`)

2 Built-in scorers:

- **Guidelines** -- Does the response follow specific rules?
- **Correctness** -- Is the answer factually correct given ground truth?
- **Safety** -- Is the response safe and appropriate?
- **RetrievalGroundedness** -- Is the response grounded in retrieved context?

3 Custom scorers with `@scorer decorator`:

```
from mlflow.genai import scorer
@scorer
def dealer_policy_compliance(inputs, outputs):
    """Check if agent response aligns with dealer policies."""
    # Custom evaluation logic
    return {"score": 0.95, "justification": "Response correctly
cited policy section 4.2"}
```

4 Regression detection -- Compare agent versions using named evaluation runs

CDK Exercise:

- Evaluate an agent that answers lender eligibility questions
- Measure retrieval groundedness against dealer policy documents
- Compare two agent versions using MLflow experiment tracking

Module 5: Monitoring in Production (15 min -- Lecture)

What to Monitor:

| Signal | Tool | Description | |-----|-----|-----| | Latency & throughput | Inference Tables | Auto-logged request/response data | | Quality drift | Lakehouse Monitoring | Detect degradation in scorer metrics | | Token usage | Trace Analysis | Cost optimization via `patterns-trace-analysis` | | Error rates | Inference Tables | Failed requests, timeout patterns |

Alerting:

- Set up SQL Alerts on inference table metrics (e.g., p95 latency > 5s)
- Use Databricks Jobs for scheduled evaluation runs against production traces

Lesson 1 Recap -- Key Takeaways

- 1 **Agent = LLM + Tools + Governance** -- Databricks provides the full stack
- 2 **ResponsesAgent** is the modern pattern (MLflow 3) for building agents
- 3 **UC Functions + Vector Search** are the primary tool types
- 4 **Always deploy via async jobs** -- synchronous deployment times out
- 5 **Evaluate early, evaluate often** -- `mlflow.genai.evaluate()` with built-in and custom scorers
- 6 **Monitor with inference tables** -- Production observability is built-in

Lesson Plan 2: Using the AI Dev Toolkit Effectively -- Creating Pipelines and Agent Bricks

Duration: 3 hours (with breaks) **Format:** Live Demo + Guided Workshop **Audience:** Data Engineers, Analytics Engineers, Solution Architects **Prerequisites:** Cursor IDE installed, Databricks workspace access, Databricks MCP server configured

Learning Objectives

By the end of this session, participants will be able to:

- 1 Use the AI Dev Toolkit (Cursor + MCP tools) to rapidly build Databricks assets
 - 2 Create Spark Declarative Pipelines (SDP) using natural language + code generation
 - 3 Build Agent Bricks: Knowledge Assistants, Genie Spaces, and Multi-Agent Supervisors
 - 4 Deploy end-to-end solutions using Asset Bundles with multi-environment support
 - 5 Understand the complete data-to-agent workflow using the toolkit
-

Module 1: Introduction to the AI Dev Toolkit (20 min -- Lecture + Demo)

What is the AI Dev Toolkit?

The AI Dev Toolkit is a Cursor IDE-based development experience that combines:

- **Cursor AI assistant** -- Natural language code generation and editing
- **Databricks MCP tools** -- Direct integration with Databricks APIs for pipelines, jobs, dashboards, agent bricks, and more
- **Agent Skills** -- Pre-built knowledge modules that teach the AI assistant Databricks best practices

Key Cursor Skills Available:

| Skill | What It Does | |-----|-----| | **synthetic-data-generation** | Generate realistic test data with Faker + Spark | | **spark-declarative-pipelines** | Create SDP pipelines (bronze/silver/gold) | | **agent-bricks** | Build Knowledge Assistants, Genie Spaces, Multi-Agent Supervisors | | **model-serving** | Deploy agents to serving endpoints | | **mlflow-evaluation** | Evaluate agent quality with scorers | | **asset-bundles** | Package everything for multi-environment deployment | | **aibi-dashboards** | Create AI/BI dashboards | | **databricks-genie** | Build and query Genie Spaces for natural language SQL | | **unstructured-pdf-generation** | Generate synthetic PDFs for RAG use cases |

Demo: The MCP Tool Ecosystem

Walk through how MCP tools connect Cursor to Databricks:

- `execute_sql` -- Run queries on a SQL warehouse
 - `create_or_update_pipeline` -- Create and run SDP pipelines
 - `create_or_update_ka` -- Build Knowledge Assistants
 - `create_or_update_genie` -- Build Genie Spaces
 - `create_or_update_mas` -- Build Multi-Agent Supervisors
 - `upload_folder` -- Push local files to Databricks workspace
 - `get_table_details` -- Inspect Unity Catalog table schemas
 - `run_python_file_on_databricks` -- Execute Python on a cluster
-

Module 2: Creating Spark Declarative Pipelines (50 min -- Live Demo)

Scenario: Build a product feedback pipeline for CDK Global that ingests raw dealer feedback data, cleans it, and

produces gold-layer analytics tables.

Step 1: Generate Synthetic Data (10 min)

Using the synthetic-data-generation skill:

Prompt Cursor:

```
"Generate synthetic dealer product feedback data for CDK Global. Include tables for dealers, feedback submissions, and product categories. Save to the ai_dev_kit.cdk_demo schema as parquet in a raw_data volume."
```

The toolkit will:

- 1 Create a Python script with Faker-generated realistic data
- 2 Use non-linear distributions (log-normal for ratings, weighted categorical for categories)
- 3 Include time-based patterns (weekday/weekend, seasonality)
- 4 Save raw parquet files to /Volumes/ai_dev_kit/cdk_demo/raw_data/
- 5 Execute on a Databricks cluster via run_python_file_on_databricks

Key Principles (from skill):

- Raw data only -- no pre-aggregated fields (pipeline handles that)
- Last 6 months of data for realistic demo windows
- 10K-50K rows minimum so patterns survive GROUP BY
- Pandas for generation, Spark for saving

Step 2: Initialize the Pipeline Project (10 min)

Option A: Asset Bundle initialization (recommended for production)

```
databricks pipelines init
# Project name: cdk_product_feedback_pipeline
# Catalog: ai_dev_kit
# Personal schema: yes (for dev)
# Language: SQL
```

This creates:

```
cdk_product_feedback_pipeline/
... databricks.yml           # Multi-environment config
... resources/
.   ... *_etl.pipeline.yml    # Pipeline resource definition
... src/
    ... *_etl/
        ... explorations/     # Exploratory notebooks
        ... transformations/  # Your SQL/Python files
```

Option B: Manual MCP workflow (for rapid prototyping)

Create pipeline files locally and use upload_folder + create_or_update_pipeline.

Step 3: Write Pipeline Transformations (20 min)

Bronze Layer -- Ingest raw data:

```
-- bronze_dealer_feedback.sql
CREATE OR REFRESH STREAMING TABLE bronze_dealer_feedback
CLUSTER BY (submission_date)
```

```

AS
SELECT
    *,
    current_timestamp() AS _ingested_at,
    _metadata.file_path AS _source_file
FROM read_files(
    '/Volumes/ai_dev_kit/cdk_demo/raw_data/feedback/',
    format => 'parquet'
);

```

Silver Layer -- Clean and validate:

```

-- silver_dealer_feedback.sql
CREATE OR REFRESH MATERIALIZED VIEW silver_dealer_feedback
AS
SELECT
    feedback_id,
    dealer_id,
    product_category,
    TRIM(feedback_text) AS feedback_text,
    CAST(rating AS INT) AS rating,
    submission_date,
    _ingested_at
FROM STREAM(bronze_dealer_feedback)
WHERE feedback_id IS NOT NULL
    AND rating BETWEEN 1 AND 5;

```

Gold Layer -- Aggregate for analytics:

```

-- gold_feedback_summary.sql
CREATE OR REFRESH MATERIALIZED VIEW gold_feedback_summary
AS
SELECT
    product_category,
    DATE_TRUNC('MONTH', submission_date) AS month,
    COUNT(*) AS total_feedback,
    AVG(rating) AS avg_rating,
    COUNT(CASE WHEN rating <= 2 THEN 1 END) AS negative_count,
    COUNT(CASE WHEN rating >= 4 THEN 1 END) AS positive_count
FROM silver_dealer_feedback
GROUP BY product_category, DATE_TRUNC('MONTH', submission_date);

```

Key SDP Best Practices (from skill):

- Use CLUSTER BY (Liquid Clustering), never PARTITION BY
- Raw .sql files, not notebooks
- Serverless compute only (default)
- read_files() for ingestion from Volumes
- Unqualified table names for pipeline-internal references (portable across environments)

Step 4: Deploy and Run (10 min)

Using MCP tools:

```

create_or_update_pipeline(
    name="cdk_product_feedback_pipeline",

```

```

root_path="/Workspace/Users/user@example.com/cdk_product_feedback_pipeline",
    catalog="ai_dev_kit",
    schema="cdk_demo",
    workspace_file_paths=[
        ".../bronze_dealer_feedback.sql",
        ".../silver_dealer_feedback.sql",
        ".../gold_feedback_summary.sql"
    ],
    start_run=True,
    wait_for_completion=True,
    full_refresh=True
)

```

Or using Asset Bundles:

```

databricks bundle deploy
databricks bundle run cdk_product_feedback_etl

```

Verify output:

```

get_table_details(
    catalog="ai_dev_kit",
    schema="cdk_demo",
    table_names=["bronze_dealer_feedback",
        "silver_dealer_feedback", "gold_feedback_summary"]
)

```

Module 3: Building Agent Bricks (50 min -- Live Demo)

The Three Agent Bricks:

Brick Purpose Data Source When to Use	----- ----- ----- -----	Knowledge Assistant (KA)
Document-based Q&A using RAG PDF/text files in Volumes Policy docs, manuals, guides		Genie Space
Natural language to SQL Unity Catalog tables Analytics, reporting, exploration		Multi-Agent Supervisor (MAS)
Multi-agent orchestration KA + Genie + Custom endpoints Compound use cases		

Demo 1: Knowledge Assistant -- Dealer Policy Q&A (15 min)

Scenario: Create a KA that answers questions about CDK's dealer onboarding policies.

Step 1: Generate synthetic PDF documents

Using the unstructured-pdf-generation skill, create realistic policy documents and upload them to a Volume:

```

/Volumes/ai_dev_kit/cdk_demo/raw_data/dealer_policies/
... dealer_onboarding_guide.pdf
... lender_partnership_policies.pdf
... compliance_requirements.pdf
... data_security_standards.pdf
... rag_queries_dealer_policies.json    # Auto-generated Q&A pairs

```

Step 2: Create the Knowledge Assistant


```

create_or_update_ka(
    name="CDK Dealer Policy Assistant",

    volume_path="/Volumes/ai_dev_kit/cdk_demo/raw_data/dealer_policies",
    description="Answers questions about CDK dealer onboarding
policies and compliance requirements",
    instructions="You are an expert on CDK Global's dealer
policies. Always cite specific policy sections. If unsure, say so
rather than guessing.",
    add_examples_from_volume=True # Auto-loads Q&A pairs from JSON
)

```

Step 3: Wait for provisioning and verify

- Endpoint status progresses: PROVISIONING --> ONLINE (2-5 minutes)
- Examples are auto-loaded from the companion JSON files
- Test via `query_serving_endpoint` or the Databricks UI

Demo 2: Genie Space -- Dealer Feedback Analytics (15 min)

Scenario: Create a Genie Space that lets business users explore dealer feedback data with natural language.

Step 1: Inspect tables (built in Module 2)

```

get_table_details(
    catalog="ai_dev_kit",
    schema="cdk_demo",
    table_names=["silver_dealer_feedback", "gold_feedback_summary"]
)

```

Step 2: Create the Genie Space

```

create_or_update_genie(
    display_name="CDK Dealer Feedback Analytics",
    table_identifiers=[
        "ai_dev_kit.cdk_demo.silver_dealer_feedback",
        "ai_dev_kit.cdk_demo.gold_feedback_summary"
    ],
    description="Explore dealer product feedback data. Ask about
ratings, trends, categories, and dealer-level insights.",
    sample_questions=[
        "What is the average rating by product category this
quarter?",
        "Which dealers submitted the most negative feedback last
month?",
        "Show me the monthly trend of feedback volume by category",
        "What percentage of feedback is negative (rating <= 2)?",
        "Which product category has the highest average rating?"
    ]
)

```

Step 3: Test with the Conversation API

```

result = ask_genie(
    space_id="<space_id>",

```

```

        question="What were the top 5 product categories by average
rating last month?"
    )
    # Returns: SQL query, columns, data rows, natural language summary

```

Genie Best Practices:

- Include 5-10 sample questions that reference actual column names
- Use gold-layer tables for better query performance
- Add instructions via the Databricks UI to refine query generation

Demo 3: Multi-Agent Supervisor -- CDK Support Hub (20 min)

Scenario: Create a MAS that routes user queries to either the policy KA or the feedback Genie Space.

Step 1: Look up existing bricks

```

# Find the KA
ka_info = find_ka_by_name("CDK_Dealer_Policy_Assistant")
# Returns: tile_id, endpoint_name, endpoint_status
# Find the Genie Space (from creation output)
# space_id from the create_or_update_genie result

```

Step 2: Create the Multi-Agent Supervisor

```

create_or_update_mas(
    name="CDK Support Hub",
    agents=[
        {
            "name": "policy_agent",
            "ka_tile_id": ka_info["tile_id"],
            "description": "Answers questions about dealer
onboarding policies, compliance requirements, lender partnerships,
and data security standards. Use for policy lookups and procedure
questions.",
        },
        {
            "name": "feedback_analytics_agent",
            "genie_space_id": "<genie_space_id>",
            "description": "Answers data questions about dealer
product feedback, ratings, trends, and category analytics. Use for
quantitative questions about feedback metrics."
        }
    ],
    description="Routes CDK support queries to the appropriate
specialized agent",
    instructions="Route policy and procedure questions to
policy_agent. Route data and analytics questions about feedback,
ratings, and trends to feedback_analytics_agent. If unclear, ask
the user to clarify.",
    examples=[
        {
            "question": "What are the requirements for dealer
onboarding?",
            "guideline": "Route to policy_agent -- this is a policy
question"

```

```

    },
    {
      "question": "What is the average feedback rating this
quarter?",
      "guideline": "Route to feedback_analytics_agent -- this
is a data analytics question"
    },
    {
      "question": "How many dealers gave negative feedback on
the new DMS product?",
      "guideline": "Route to feedback_analytics_agent -- this
is a quantitative data question"
    }
  ]
)

```

Step 3: Test routing

```

query_serving_endpoint(
  name="cdk-support-hub-endpoint",
  messages=[{"role": "user", "content": "What compliance
documents do new dealers need to submit?"}]
)
# --> Routes to policy_agent (Knowledge Assistant)
query_serving_endpoint(
  name="cdk-support-hub-endpoint",
  messages=[{"role": "user", "content": "Show me average ratings
by product category for Q4"}]
)
# --> Routes to feedback_analytics_agent (Genie Space)

```

Module 4: Packaging with Asset Bundles (20 min -- Lecture + Demo)

Why Asset Bundles?

- Multi-environment deployment (dev/staging/prod)
- Version-controlled infrastructure as code
- Reproducible, auditable deployments
- CI/CD integration with GitHub Actions

Bundle Structure for CDK:

```

cdk_product_feedback/
... databricks.yml           # Main config + targets
... resources/
.   ... pipeline.yml         # SDP pipeline definition
.   ... dashboard.yml        # AI/BI dashboard
.   ... job.yml              # Scheduled jobs
.   ... app.yml              # Databricks App (optional)
... src/
.   ... pipelines/
.     ... bronze_dealer_feedback.sql
.     ... silver_dealer_feedback.sql
.     ... gold_feedback_summary.sql

```

```

... agent_bricks.py          # KA + Genie + MAS setup
... app/                     # Web application (optional)
...   app.py
...   app.yaml

```

Key Bundle Configuration:

```

# databricks.yml
bundle:
  name: cdk-product-feedback
variables:
  catalog:
    default: "ai_dev_kit"
  schema:
    default: "cdk_demo"
targets:
  dev:
    default: true
    mode: development
    variables:
      catalog: "ai_dev_kit"
      schema: "cdk_demo_dev"
  prod:
    mode: production
    variables:
      catalog: "cdk_prod"
      schema: "product_feedback"

```

Deployment Commands:

```

databricks bundle validate      # Validate configuration
databricks bundle deploy       # Deploy to dev (default)
databricks bundle deploy -t prod # Deploy to production
databricks bundle run pipeline_name # Run the pipeline

```

Module 5: Hands-on Workshop (40 min)

Exercise: Using the AI Dev Toolkit, each participant will:

- 1 **Generate data** (10 min) -- Use Cursor to prompt for synthetic data relevant to their team's domain
- 2 **Build a pipeline** (15 min) -- Create a 3-layer SDP pipeline from the raw data
- 3 **Create an Agent Brick** (15 min) -- Choose one:
 - Knowledge Assistant from synthetic PDFs
 - Genie Space from the pipeline output tables
 - Multi-Agent Supervisor combining both

Suggested Prompts for Cursor:

For data generation:

"Generate synthetic dealer service appointment data for CDK Global with 15,000 appointments over the last 6 months. Include dealer_id, customer_name, service_type, vehicle_make, vehicle_model, appointment_date, duration_minutes, and satisfaction_score."

For pipeline creation:

```
"Create a Spark Declarative Pipeline that ingests the dealer service appointment data from the raw_
data volume, cleans it in silver, and produces a gold summary with average satisfaction by service type and dealer.
"
```

For Genie Space:

```
"Create a Genie Space called 'CDK Service Analytics' that lets users explore dealer service
appointment data. Include sample questions about appointment volume, satisfaction trends, and
popular service types."
```

Lesson 2 Recap -- Key Takeaways

- 1 **The AI Dev Toolkit accelerates everything** -- Natural language prompts generate production-quality Databricks assets
 - 2 **Pipelines follow the medallion pattern** -- Bronze (ingest) --> Silver (clean) --> Gold (aggregate) using SDP
 - 3 **Agent Bricks are composable** -- KA for documents, Genie for SQL, MAS to combine them
 - 4 **Asset Bundles = production readiness** -- Multi-environment, version-controlled, CI/CD-compatible
 - 5 **The workflow is iterative** -- Generate data --> build pipeline --> create bricks --> test --> refine
-

Appendix A: Environment Setup Checklist

Before the sessions, ensure each participant has:

- ☐ Cursor IDE installed (latest version)
- ☐ Databricks CLI installed (`pip install databricks-cli`)
- ☐ Databricks MCP server configured in Cursor settings
- ☐ Access to a Databricks workspace with Unity Catalog enabled
- ☐ Access to a running SQL Warehouse
- ☐ Access to a running compute cluster (for Python execution)
- ☐ `ai_dev_kit` catalog created (or equivalent)
- ☐ Cursor skills installed (agent-bricks, spark-declarative-pipelines, etc.)

Appendix B: Reference Links

| Resource | Link | |-----|-----| | Lakeflow Spark Declarative Pipelines | <https://docs.databricks.com/aws/en/ldp/> | | MLflow 3 ResponsesAgent | <https://mlflow.org/docs/latest/llms/responses-agent-intro/> | | Agent Framework | <https://docs.databricks.com/generative-ai/agent-framework/> | | Model Serving | <https://docs.databricks.com/machine-learning/model-serving/> | | Asset Bundles | <https://docs.databricks.com/dev-tools/bundles/> | | Unity Catalog | <https://docs.databricks.com/data-governance/unity-catalog/> | | Genie Spaces | <https://docs.databricks.com/en/genie/> | | MLflow GenAI Evaluation | <https://mlflow.org/docs/latest/llms/llm-evaluate/> |

Appendix C: Post-Session Follow-Up

Week 1: Participants replicate the demo using their own team's data domain **Week 2:** Office hours for troubleshooting and advanced questions **Week 3:** Review deployed assets, discuss production readiness and

governance **Week 4:** Present team solutions and identify next steps for production rollout